

Article

ASISTENCIAS UAGro: An Integrated System for Faculty Attendance Management and Control at the Faculty of Engineering

Angel Morales Chazari ^{1,*} 

¹ Facultad de Ingeniería, Universidad Autónoma de Guerrero (UAGro), Chilpancingo, Guerrero, México; autor@uagro.mx

Abstract

This project presents ASISTENCIAS UAGro, a comprehensive technological solution for modernizing and automating faculty attendance registration at the Faculty of Engineering of the Autonomous University of Guerrero (UAGro). The system integrates a mobile application for field data capture and an administrative web portal for centralized management, applying software engineering principles that ensure efficiency, accuracy, and usability. The solution addresses the limitations of the manual registration process, which is prone to errors and consumes administrative time. The mobile application facilitates agile attendance registration, schedule checks, and viewing building status through an interactive map. The web portal allows for data management, report generation, permit management, and record correction, with real-time synchronization through the Firebase Realtime Database. ASISTENCIAS UAGro represents a case study in full-stack development, encompassing UI/UX design, mobile programming, web development, and integration with cloud services.

Keywords: Management System; Faculty Attendance; Mobile Application; Firebase; Full-Stack Development; Automation; UAGro

1. Introduction

The management of faculty attendance is a fundamental process within academic administration, as it directly impacts the organization, evaluation, and monitoring of educational performance. However, in many institutions, this process is performed manually, which leads to delays, errors in records, and difficulties in accessing accurate and timely historical information. These limitations affect administrative efficiency and the quality of academic oversight, creating a need for technological solutions that can optimize and modernize this task.

In this context, ASISTENCIAS UAGro was developed, a comprehensive system that combines a mobile application and a web platform to automate and facilitate the registration and control of faculty attendance at the Faculty of Engineering of the Autonomous University of Guerrero. The mobile application is designed for the supervisors in charge of on-site verification, providing tools to register attendance quickly, consult schedules, and view the status of buildings through intuitive visual indicators. This functionality significantly reduces the time spent on data entry and improves the organization of the information collected.

On the other hand, the web platform is designed exclusively for the system administrator, offering a robust and complete environment for the centralized management of

Received:

Accepted:

Published:

Copyright: © 2026 by the author.

Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the

[Creative Commons Attribution \(CC BY\)](#) license.

information. This module allows for querying and modifying attendance records, accessing data history, uploading new information, and managing permissions for faculty. Furthermore, it incorporates the generation of reports in PDF format, grouped into compressed files to facilitate their storage and distribution. Real-time data synchronization, using Firebase Realtime Database, guarantees consistency, constant updates, and the traceability of information.

Together, ASISTENCIAS UAGro represents a technological solution that not only improves operational efficiency and accuracy in attendance control but also strengthens the transparency and reliability of academic records. This project contributes to the modernization of university administrative processes and offers a relevant case study in the development of *full-stack* applications, integrating interface design, mobile programming, web development, and cloud services.

2. Materials and Methods

2.1. Methodology

The architecture of ASISTENCIAS UAGro is based on a modular model that integrates two main components: a mobile application developed in Flutter for field data capture and visualization, and an administrative web portal for centralized management. Communication and synchronization between both components is handled via Firebase Realtime Database, which acts as a real-time *backend*, guaranteeing the consistency and immediate availability of information.

The workflow begins with the retrieval of class, schedule, and building data from Firebase, which feeds dynamic filters in the mobile application to segment queries by day, shift, building, and time. Attendance registration is performed in real time, storing discrete states (attended, absent, pending) under unique keys that combine the professor's identifier and the class schedule.

For state management, the system classifies classes as pending, reviewed, and unsupervised, establishing a 15-minute time window after the start of each session to allow for registration. The daily update is managed by a periodic timer that detects the day change, clearing caches and internal lists to prevent the persistence of outdated data.

This modular, cloud-service-based architecture enables an efficient, scalable, and easily maintainable system, optimizing attendance supervision and adapting to the operational needs of the Faculty of Engineering.

2.2. Resource Engineering

The ASISTENCIAS UAGro system was designed to meet a set of functional and non-functional requirements that ensure its efficient and reliable operation for faculty attendance control.

2.2.1. Functional Requirements

The system must allow the real-time registration of faculty attendance, clearly differentiating between the states of "attended," "absent," and "pending." To facilitate queries, dynamic filters are implemented to segment classes by day, shift, building, and time, optimizing the search for relevant information. The review status is visualized through intuitive visual indicators that reflect the state of each classroom and building.

The administrative web portal provides functionalities for centralized management, including the correction of records, permission administration, and consultation of the attendance history. Furthermore, the system generates reports in PDF format, grouped into compressed files to facilitate their storage and distribution. Real-time synchroniza-

tion between the mobile application and the web portal guarantees data consistency and immediate availability.

To improve supervision, the system automatically classifies classes as pending, reviewed, and unsupervised, considering a 15-minute time window after the start of each session for attendance registration. Likewise, an automatic daily update mechanism is implemented that clears obsolete data and maintains the integrity of the displayed information.

2.2.2. Non-Functional Requirements

The mobile application must be cross-platform, compatible with Android and iOS devices, to maximize its accessibility. The user interface was designed to be intuitive and user-friendly, minimizing the learning curve for supervisors and administrators.

Security and data privacy are guaranteed through appropriate controls that protect sensitive information. The system is scalable, allowing for the incorporation of new users and the handling of a growing volume of data without affecting performance.

Data synchronization and updating is performed efficiently, ensuring that information is always available and updated in real time. Finally, the solution is robust and fault-tolerant, minimizing data loss and ensuring operational continuity.

2.3. *Mock-up (Interface Design)*

The mock-up for ASISTENCIAS UAGro was conceived from its initial stages with a strategic approach aimed at facilitating agile and efficient development, as well as providing a clear, intuitive, and functional user experience. To this end, visual simplicity and the logical organization of elements were prioritized, seeking to minimize complexity in both implementation and user interaction.

In the mobile application, standard and widely supported components were selected to allow for rapid integration and maintenance, such as drop-down menus for filter selection (shift, building, time, and day). These filters dynamically segment information, facilitating specific queries and reducing visual overload. The interface structure uses expandable elements that reveal additional details only when requested by the user, optimizing screen space usage and avoiding information saturation.

Class lists are organized in clearly differentiated sections according to their state (pending, reviewed, unsupervised), with fixed headers that enhance navigation and readability on screens with large volumes of data. Color coding for attendance states (green, red, and transparent) allows for quick and efficient identification, decreasing cognitive load and facilitating decision-making.

The design of the administrative web portal focused on maximizing efficiency in centralized management, through intuitively organized panels and forms that allow for easy consultation, modification, and administration of records, permissions, and reports. The arrangement of elements follows principles of usability and accessibility, ensuring the administrator can execute tasks quickly and without complications. Furthermore, the adaptability of the interfaces for different devices and resolutions was considered, ensuring a consistent and accessible experience. The choice of a modular and reusable design contributed to accelerating development and facilitating future expansions or modifications to the system.

The combination of all these considerations makes the ASISTENCIAS UAGro mock-up represent a balance between functionality and simplicity, which not only facilitates development and maintenance but also ensures that the interfaces effectively meet the operational and usability needs of the end users.

3. Data Modeling

The design of the data model is a fundamental pillar in the architecture of ASISTENCIAS UAGro, as it defines how the system's critical information is structured, stored, and retrieved. An appropriate choice in this area is crucial to guarantee operational efficiency, future scalability, and data integrity in a real-time environment. This section details the technical decisions and justifications behind the implemented data schema.

3.1. Principles and Objectives of Modeling in ASISTENCIAS UAGro.

Data modeling in ASISTENCIAS UAGro was designed with the goal of optimizing the storage, query, and synchronization of information related to faculty attendance, ensuring efficiency, scalability, and consistency. Given that the system requires handling real-time data, with multiple filters and states, a model that facilitates rapid access and immediate updates was prioritized, minimizing latency and complexity in operations. The approach was oriented towards a schema that allows representing the relationships between key entities—such as professors, classes, schedules, buildings, and attendance statuses—in a way that facilitates the aggregation and segmentation of data for filtering and reporting functionalities.

These components that together constitute the application are:

Professor: The Professor component represents the faculty in the system, uniquely identified by the `profeID` attribute and accompanied by their full name `nombre`. This identification is essential for constructing the unique attendance registration key, guaranteeing the traceability and uniqueness of each class taught. The one-to-many relationship with the Class entity reflects that a professor can teach multiple classes.

Class: Contains the specific details of each academic session, including attributes such as `aula` (classroom identifier), `horario` (class time slot), `dia` (day of the week), `grupo` (assigned section) and `materia` (subject name), as shown in Figure 2. The unique key `claveRegistro` is generated by concatenating `profeID` with `horario`, which facilitates efficient indexing and querying in Firebase. This structure allows for the implementation of dynamic filters and groupings by building or shift, optimizing the user experience. The relationship with Building indicates the physical location of the classroom.

Building: Represents the physical location where the classrooms are. Each class is associated with a building, identified by its name, which allows segmenting information and applying specific filters in the user interface. This association is key for the organization and visualization of the data, as reflected in the implemented filtering logic.

Attendance: This entity stores the attendance status for each class, with attributes such as `estado` (e.g., attended, absent, pending) and `timestamp` to record the exact moment of registration. Additionally, it includes geolocation information and data from the supervisor who registers the attendance, represented by the `Ubicacion` and `Supervisor` entities, respectively. This structure guarantees the integrity and validity of the records, allowing verification that attendance was registered within the permitted temporal and spatial range.

Location: Contains the geolocation data associated with each attendance record, such as latitude, longitude, address, accuracy, distance to the school, and whether the registration was made within the permitted radius. This data is captured through integration with geolocation services in the mobile application, ensuring the validity and reliability of the

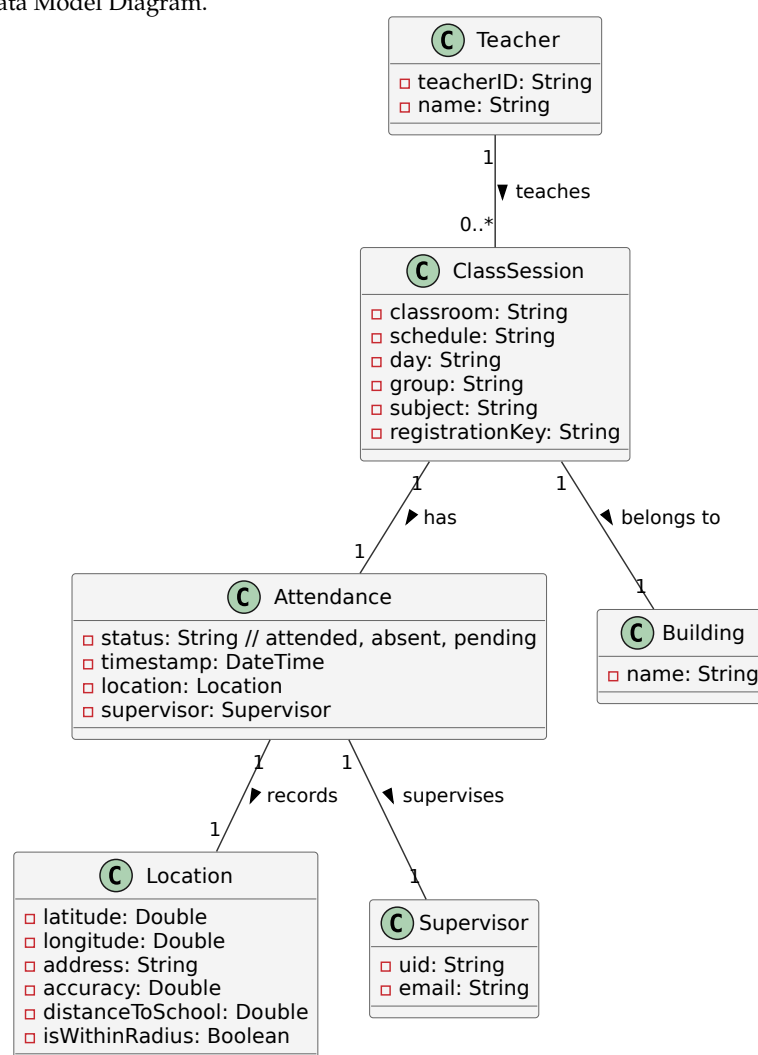
registration.

Supervisor: Finally, the Supervisor entity represents the user who validates and registers the attendance, with attributes such as *uid* and *email*. This information is essential for audit and control within the system, allowing identification of who performed each registration.

By integrating these components—Professor, Class, Building, Attendance, Location, and Supervisor—a robust and coherent data model is achieved, specifically designed to respond to the real-time management needs of the system. Each entity provides essential attributes that guarantee the traceability, integrity, and reliability of the records, while their relationships reflect the operational dynamics of the Faculty of Engineering.

Collectively, this conceptual structure is integrally represented in Figure 1, which visualizes the entities and their interconnections within the ASISTENCIAS UAGro data model. This diagram serves as the basis for understanding how the organization of information supports critical processes such as class filtering, attendance validation, and real-time synchronization, naturally leading to the justification of the selected technology for its implementation.

Figure 1. Data Model Diagram.

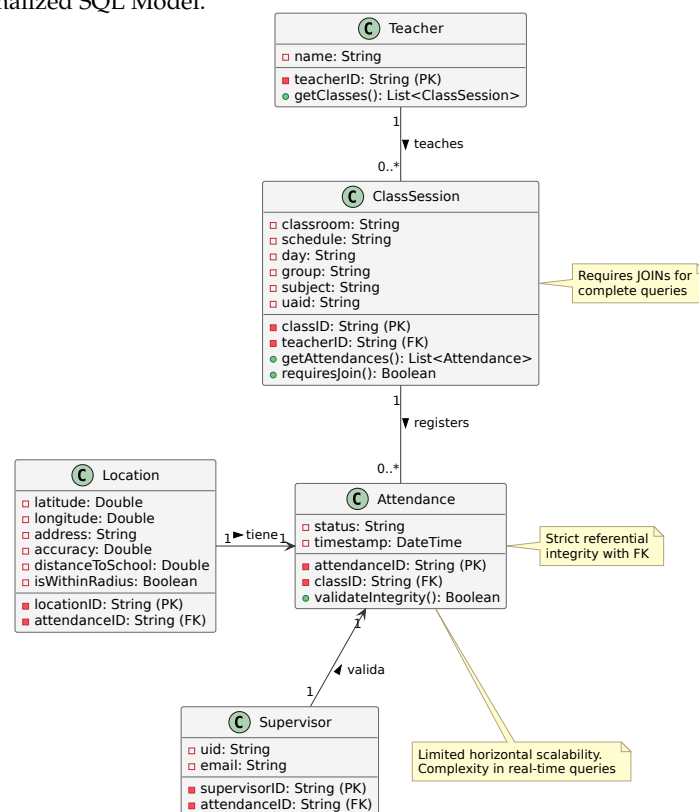


3.2. Selected Technology and Justification

For the data modeling and storage in ASISTENCIAS UAGro, **Firestore Realtime Database** was chosen, a NoSQL document-oriented database optimized for real-time applications. This decision is based on several key technical and operational factors detailed below.

In the relational (SQL) model, entities are normalized, meaning information is distributed across multiple tables related to each other via primary keys (PK) and foreign keys (FK). For example, the Professor entity stores the faculty member's data and is related to the Class entity through the `profeID` attribute. In turn, Class is related to Attendance, where the participation status of each session is recorded along with details like geolocation and supervisor. This approach guarantees referential integrity and avoids information redundancy, but increases query complexity, as it requires join operations to obtain complete information, which can affect performance in systems demanding real-time responses, as seen in Figure 2.

Figure 2. Normalized SQL Model.

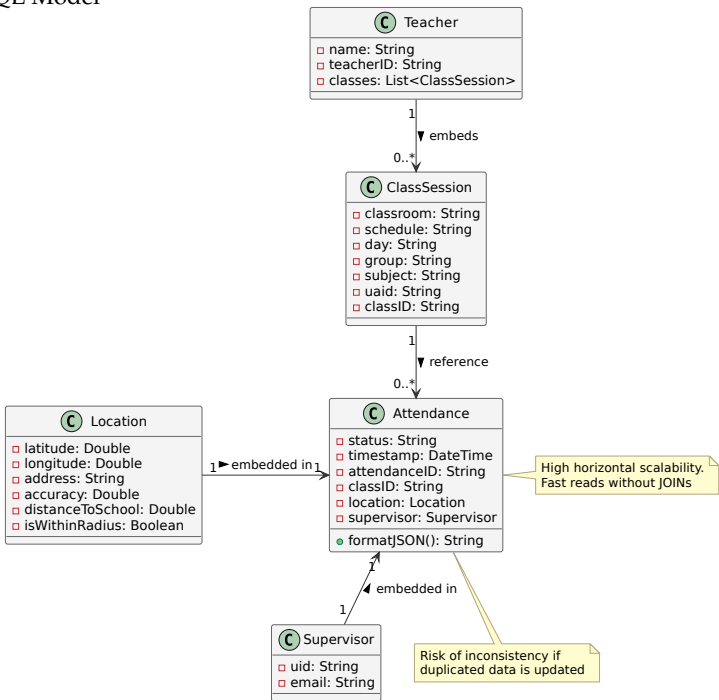


In contrast to this, the NoSQL model of **Firestore Realtime Database** organizes data in a hierarchical and denormalized structure, where entities are nested in JSON nodes. In this way, direct access to information is facilitated through dynamically generated unique keys, such as the concatenation `profeID_hora`, which exclusively identifies each attendance record as shown in Figure 3. Additionally, this model was selected because it offers native integration with Flutter and other Google Cloud services, which allows significantly reducing development complexity and deploying new functionalities faster.

This model is designed to minimize dependency on cross-queries and eliminate the need for joins, optimizing rapid access, immediate updates, and real-time synchronization

between client and server. Another key advantage is its ability to manage offline states, ensuring the application remains functional in scenarios with intermittent connectivity and synchronizing changes as soon as the connection is restored. Although it sacrifices some strict integrity features present in SQL, this approach offers greater efficiency and horizontal scalability, especially in mobile applications and systems with high concurrency. Likewise, Firebase incorporates advanced security measures and integrated authentication, guaranteeing both information protection and system reliability under high load levels.

Figure 3. NoSQL Model



In summary, while the SQL model prioritizes strong consistency through normalization, the NoSQL (Firebase) model prioritizes availability and access speed, providing an ideal balance between structural simplicity and performance for the ASISTENCIAS UAGro use case.

3.3. Model Orientation Towards the Application

The previously defined data model does not remain solely as a conceptual or technical representation; its design is directly oriented to support the critical functionalities of the ASISTENCIAS UAGro application. The structure proposed in Firebase Realtime Database becomes the foundation that allows each action performed by the faculty member, supervisor, or the system itself to translate into efficient and reliable operations. This interaction is illustrated in Figure 4, which breaks down the system into its macro-architectural flow (Figure 4a) and its internal NoSQL data scheme (Figure 4b).

From the end-user perspective, this model enables functions such as: automated attendance registration, real-time location validation, and filtering of classes by classroom, building, or day. These operations are executed transparently thanks to the node hierarchy in the database, where the linkage between professor, class, and attendance is optimized to minimize latency and simplify queries. A concrete example of this orientation is the unique key generated for each attendance record, based on the concatenation of attributes like `profID` and the schedule. This scheme, previously conceptualized in the data modeling,

finds its practical application here by allowing the immediate identification of a specific attendance without the need to traverse complex structures.

Similarly, the integration of sub-nodes like location and supervisor within each attendance record responds to the operational need to validate the context of the registration: not only is the professor's presence status saved, but also the confirmation of their geographic location and the identity of the person validating the report. This way of structuring the data allows the application logic to remain simple, since accessing the attendance record is sufficient to have all relevant information. On the technical side, this model also translates into an advantage for development in Flutter, given that thanks to the native integration with Firebase, data is delivered in real time via reactive streams. This feature ensures that any change in the recorded attendance, whether from the mobile app or the supervision portal, is immediately reflected in the interface, thus contributing to a fluid and reliable experience.

In short, the data model not only fulfills a structural role but is explicitly designed to support the application's operational processes, reducing code complexity, improving system response, and guaranteeing that the core functions of supervision and attendance control are executed without friction. This orientation reinforces the alignment between the technical architecture and the functional objectives of the ASISTENCIAS UAGro project.

Figure 4. (a) Macro-Architecture: Client applications interacting with the infrastructure layer.

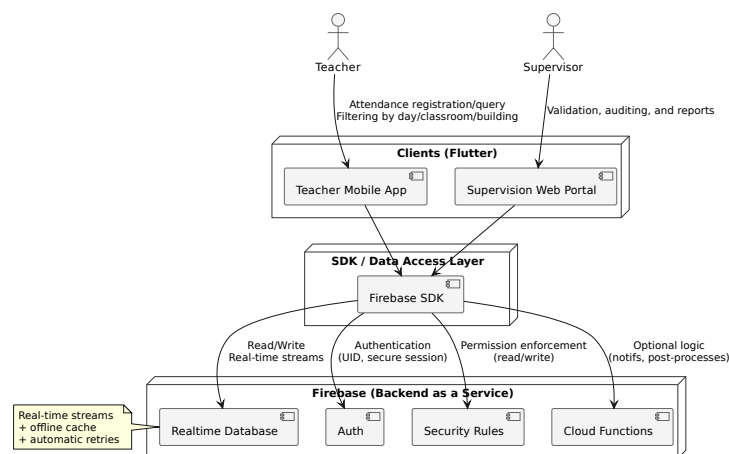
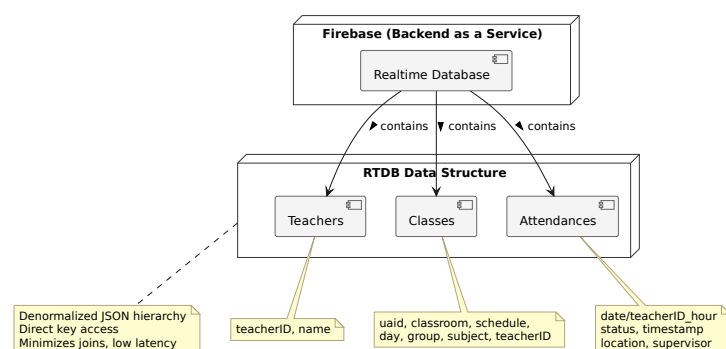


Figure 5. (b) Micro-Architecture: Hierarchical JSON data structures and sub-node embedding.



3.4. Denormalization

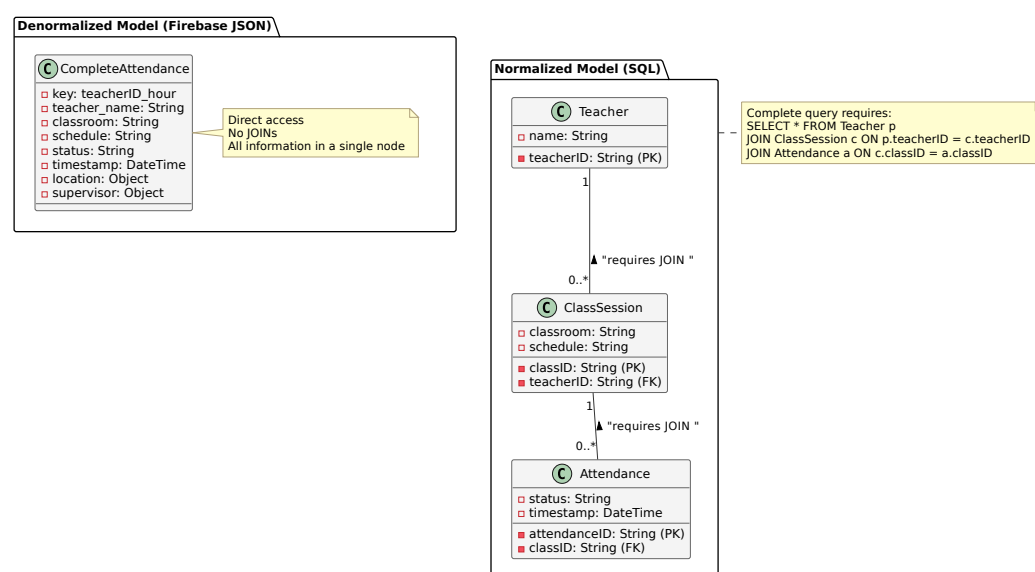
Denormalization constitutes a deliberate strategy in database design that involves repeating or grouping information in less rigid structures, with the goal of simplifying

access and optimizing performance in specific situations. Unlike the normalized schemas typical of relational databases—where each entity is distributed in separate tables and linked through primary and foreign keys—in environments that demand immediacy in operations, denormalization offers direct access and reduces query complexity.

In the case of ASISTENCIAS UAGro, the choice of Firebase Realtime Database naturally led towards a denormalized model. The hierarchical structure in JSON format allows representing different levels of information within the same node, such as the linkage between professor, class, and attendance record, without the need for join operations. This design is reflected, for example, in the generation of unique keys like **profeID_hora**, which unify in a single identifier data that, in a relational environment, would be distributed across several tables. The denormalization applied in this project responds to a concrete operational need: guaranteeing the availability of information in real time. Thanks to this decision, the system is capable of responding with minimal latency to critical operations like attendance registration and location validation, aspects that would be more costly in a strictly normalized schema. In this way, dependence on complex queries is avoided, and access speed is prioritized, favoring the user experience in both the mobile application and the supervision portal.

It is important to emphasize that this strategy does not replace rigor in design, but rather complements it by aligning it with the domain requirements. While in previous modeling phases the entities and their attributes (professor, class, attendance, location, and supervisor) were clearly defined, denormalization in Firebase ensures that these elements are available in operational structures optimized for direct querying. This achieves a balance between the conceptual clarity of a normalized model and the practical efficiency of a denormalized model, reinforcing the system's orientation towards real-time work, this difference being visually synthesized in Figure 5.

Figure 6. Comparison: Normalization vs. Denormalization



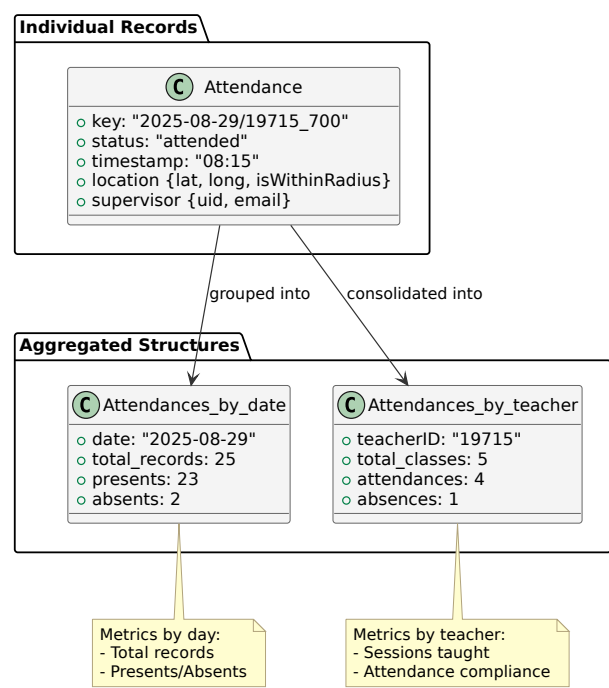
3.5. Aggregated Data

The concept of aggregated data refers to the ability to organize and present combined information from individual records, in order to facilitate analysis, visualization, and decision-making. In traditional (SQL) databases, this is achieved through aggregation queries like COUNT, SUM, or GROUP BY, which allow obtaining cumulative metrics from normalized tables. However, in the context of systems requiring immediacy in querying and real-time synchronization, such as **ASISTENCIAS UAGro**, aggregations are designed differently.

Since the system's architecture is based on Firebase Realtime Database, the aggregation strategy is not focused on executing complex queries every time data is requested, but on maintaining information structures that are already prepared or easily filterable within the JSON hierarchy. This implies that certain frequently used calculations (such as the number of attendance records for a class, a professor's compliance percentage, or the generation of daily reports) are constructed and stored incrementally at the time of registration, thereby reducing processing load in subsequent requests as illustrated in Figure 6, which shows the difference between individual records and their corresponding consolidation into aggregated structures. A concrete example of this practice is observed in attendances grouped by date and professor: by storing each record under a composite key like **fecha/profeID_hora**, it is possible to access all data for a workday with a single query, avoiding traversing multiple tables or executing multiple joins, as would happen in a relational model. This organization not only optimizes access speed but also facilitates the immediate visualization of indicators, such as the list of professors who have already registered attendance on a specific day.

Complementarily, the denormalization of the model (presented in the previous section) supports the construction of these aggregated data by maintaining in the same node information that, in an SQL system, would be distributed across different tables. In this way, the application can dynamically build reports that integrate professor, class, and location data without the need for costly intermediate operations. In summary, the aggregated data approach in **ASISTENCIAS UAGro** responds to the need to offer consolidated information in real time. By delegating part of the aggregation work to the design of the data structure itself, the system ensures that key processes like report generation, compliance validations, and metric visualization are agile and consistent, aligning with the central objective of enabling efficient and reliable management of faculty attendance.

Figure 7. Individual Data vs. Aggregated Data in Firebase RTDB



4. Implementation

The implementation stage constitutes the bridge between the system’s conceptual design and its materialization into an operational application. In the case of ASISTENCIAS UAGro, development was focused on maintaining coherence with the previously established data model, ensuring that the hierarchical structures in Firebase and the denormalization decisions were directly reflected in the code and user interfaces. This ensures that the critical functions—registration, validation, and control of faculty attendance—can be executed with the same efficiency anticipated in the modeling. The solution was conceived under a client-server approach supported by **Firestore as a Backend-as-a-Service (BaaS)**, which allowed reducing the complexity of infrastructure management and concentrating efforts on application functionality. In this way, the system’s responsibility is distributed across three main layers, as shown in Figure 8, which illustrates the general implementation architecture of the system divided into its cloud services infrastructure layout (Figure 8a) and its storage and reactive synchronization loop (Figure 8b):

A **presentation layer**, implemented in Flutter, which includes both the mobile application for faculty and the web portal for supervisors[cite: 78].

A **cloud services layer**, supported by Firebase, encompassing authentication, real-time database, security rules, and state synchronization[cite: 83, 84].

A **data model integration cycle**, which channels all attendance records, filters, and metrics through the previously defined denormalized JSON structure[cite: 105, 106].

Development focused on building reactive applications, capable of reflecting in the interface the changes occurring in the backend without the need for reloading or executing manual queries. To achieve this, **data streams**, inherent to the integration between Flutter and Firebase, were employed[cite: 97, 98]. This feature ensures that when a supervisor

registers a faculty member's attendance from the mobile application, the record is updated in the online portal. Another key aspect of the implementation was the adoption of a **secure access control** scheme, where every interaction with the database is regulated through Firebase Auth and security rules based on UID [cite: 85, 87, 89]. This not only protects the integrity of the information but also delimits the allowed operations for each actor in the system, reinforcing the platform's reliability.

In general terms, the implementation of ASISTENCIAS UAGro is distinguished by the constant alignment between design and implementation: the data model guides capture, validation, and reporting operations, while the cloud services and presentation layer guarantee a smooth and coherent experience for end users.

Figure 8. (a) Flow and Infrastructure: Presentation layer interacting with core cloud identity services.

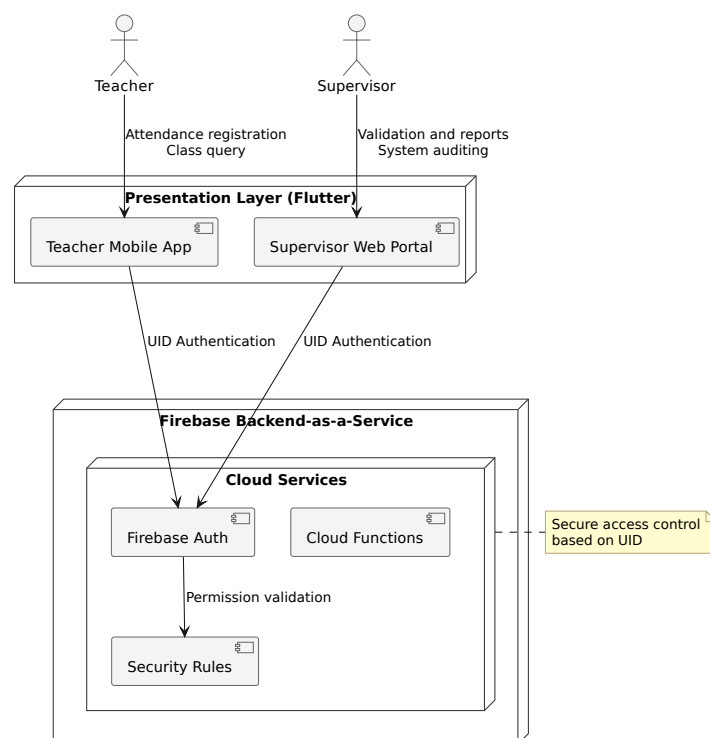
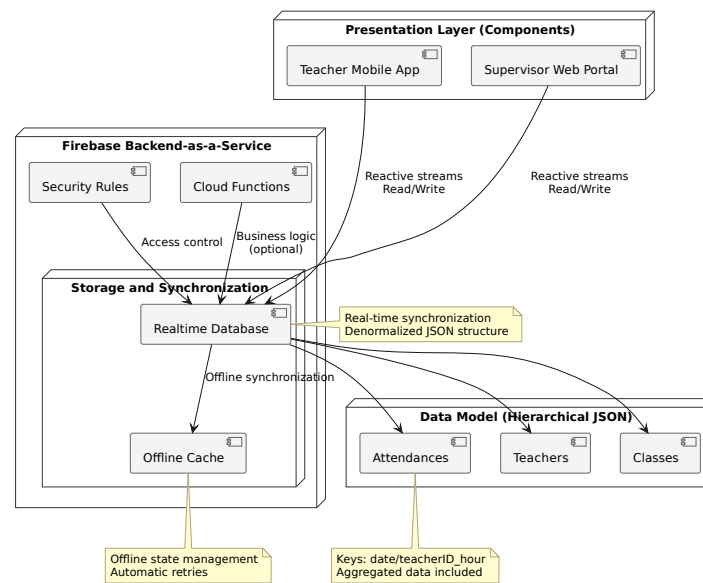


Figure 9. (b) Storage and Data Model: Reactive data streams, local offline cache sync, and hierarchical data layouts.



4.1. Frontend (Mobile App and Web Portal)

The frontend development for ASISTENCIAS UAGro was carried out using the **Flutter** framework, which allowed for a unified environment for creating both the mobile application intended for supervisors and the web portal oriented towards administrators. This technological decision ensured consistency in the user interface and facilitated the reuse of logical and visual components across different platforms. The mobile application, aimed at supervisors, concentrates the essential functions related to attendance registration and consultation of scheduled classes. Its design focuses on simplicity and speed of use, reducing interaction to a limited set of actions: registration via location validation, automatic selection of the ongoing class, and immediate confirmation of attendance status. These functionalities integrate seamlessly with the real-time database, ensuring that each record is reflected simultaneously in the supervision portal.

For its part, the web portal constitutes the specialized interface for control, audit, and report generation activities. From this platform, the administrator accesses a consolidated view of the information: lists of recorded attendances, validation of statuses, and metrics by date or professor. The interface was designed with a responsive approach, so it can be consulted not only from desktop computers but also from mobile devices in field supervision environments.

A central aspect of the frontend is the native integration with **Firestore streams**, which allow for the automatic synchronization of database changes with visual components. In this way, both the application and the portal keep their data updated without the need to reload views or manually execute queries. By relying on this reactive pattern, the system offers a fluid user experience, in which a registration generated by a supervisor is immediately visible to the corresponding administrator.

Overall, the frontend design aims to achieve a balance between functionality and efficiency, ensuring that each actor in the system interacts only with the tools and views necessary for the fulfillment of their activities. This avoids information overload and maximizes usability, consolidating the interface as a key component in the operational success of the platform.

4.2. Backend (Firebase Services)

The backend of **ASISTENCIAS UAGro** was built on the **Firebase** platform, under a *Backend-as-a-Service* (BaaS) approach. This decision reduced the infrastructure burden and concentrated development on specific application functionalities, leveraging already integrated components for authentication, database, and security.

The **authentication service (Firebase Auth)** defines each actor's access through unique identifiers. In this way, supervisors using the mobile application can only execute operations linked to their role, while administrators, from the web portal, access expanded tools such as user management and global report consultation. The core of storage relies on the **real-time database**, responsible for keeping attendance, schedule, and validation information always synchronized between clients. This means that any modification made on one device is immediately reflected in the others, without the need for reloads or additional queries.

Access security is implemented through rules associated with each user's UID. These establish who can read or modify data, delimiting actions according to their profile type and guaranteeing the integrity of the information. Finally, **Cloud Functions** complement this architecture with automatic processes, such as generating notifications or calculating frequently used metrics. Although not the central component, they are useful for extending capabilities without increasing system complexity.

Overall, the backend acts as the pillar of the application by coordinating authentication, security, and real-time synchronization, ensuring consistent operation between the mobile app and the administrative portal.

4.3. Integration with the Data Model

The implementation of **ASISTENCIAS UAGro** required a direct integration between the application's logic and the structure defined in the database. This relationship ensures that the operations performed from the mobile app or web portal consistently correspond with the model designed in Firebase.

Each interaction—from attendance registration to validation by a supervisor or a global consultation by an administrator—translates into reads or writes on specific nodes in the JSON hierarchy. Thanks to the clear definition of keys and sub-nodes, access is performed without complex intermediate queries: a composite key immediately identifies an attendance record, while associated sub-nodes concentrate additional information such as location and supervisor validation. The native integration of Flutter with Firebase simplifies this process by allowing subscription to **data streams**, which reflect in real time any change made in the database. Thus, the hierarchical structure functions not only as a repository but also as a permanent communication channel between the various system clients. The aggregated data functionality is also implemented through this integration: both daily metrics and faculty performance indicators are generated from pre-built structures in the database, which reduces computational work in the application and improves response speed in the interface.

In summary, integration with the data model turns the Firebase structure into an operational core, where every high-level operation in the application finds a direct reflection in a database node, guaranteeing coherence, immediacy, and consistency throughout the entire platform.

5. Results	446
6. Discussion	447
En esta sección puedes discutir de forma más explícita:	448
• Cómo tus resultados se comparan con trabajos previos.	449
• Ventajas y limitaciones de ASISTENCIAS UAGro.	450
• Posibles líneas de trabajo futuro (extender a otras facultades, integración con otros sistemas, analítica avanzada, etc.).	451
7. Conclusion	453
The development of ASISTENCIAS UAGro demonstrated the feasibility of a technological solution focused on academic management with criteria for efficiency, security, and real-time availability. The integration of a flexible architecture based on Firebase and Flutter enabled the creation of a cross-platform platform capable of responding immediately to the needs of faculty, supervisors, and administrators in different usage scenarios. Beyond its technical implementation, the project highlights the value of an approach focused on operational simplicity and scalability, where each user interaction is reflected in more agile, transparent, and reliable administrative processes. This lays the groundwork for the digitization of academic management in the Faculty of Engineering, offering a replicable model that can be extended to other institutional areas.	454
In summary, ASISTENCIAS UAGro achieves its central objective: to provide a comprehensive tool that optimizes the control of faculty attendance, contributing to the improvement of educational administration and strengthening trust in institutionally supported technological processes.	465
Author Contributions: Conceptualization, X.X.; methodology, X.X.; software, X.X.; validation, X.X.; formal analysis, X.X.; investigation, X.X.; resources, X.X.; data curation, X.X.; writing—original draft preparation, X.X.; writing—review and editing, X.X.; visualization, X.X.; supervision, X.X.; project administration, X.X. All authors have read and agreed to the published version of the manuscript.	469
Funding: This research received no external funding.	473
Institutional Review Board Statement: Not applicable.	474
Informed Consent Statement: Not applicable.	475
Data Availability Statement: The data presented in this study are available on request from the corresponding author.	476
Acknowledgments: Los autores agradecen el apoyo de la Facultad de Ingeniería de la Universidad Autónoma de Guerrero y de los supervisores que participaron en la evaluación del sistema.	478
Conflicts of Interest: The authors declare no conflicts of interest.	480
Abbreviations	481
The following abbreviations are used in this manuscript:	482
UAGro Universidad Autónoma de Guerrero	483
BaaS Backend-as-a-Service	484
UI/UX User Interface / User Experience	484
RTDB Realtime Database	484
References	485
1. Apellido, N.; Apellido, N. Título del artículo relacionado con sistemas de asistencia. <i>Nombre de la Revista Año, Volumen</i> , páginas.	486
	487

2. Apellido, N. Título del libro sobre bases de datos o Firebase. Editorial: Ciudad, País, Año. 488

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are 489
solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). 490
MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from 491
any ideas, methods, instructions or products referred to in the content. 492